FIAP

Challenge - Domain Driven Design Using Java

Entrega - 4º Sprint

Calculadora de CO2

API RESTful com Spring Boot e Oracle

Nome da solução: Calculadora de CO2 (API RESTful)

Nome da equipe: EcoSoul

Integrante	RM
Thiago Andrade Silva Piedade	569741
Giovanni Lopez Zavam	571403
Arthur Palacio Alves	573441
Murilo Martins de Campos	572674

Repositório GitHub	https://github.com/EuThiaguera/CalculadoraCO2
Deploy (Render)	https://calculadoraco2.onrender.com/calculos

1. Objetivo e escopo do projeto

A **Calculadora de CO2** é uma API RESTful que registra trajetos feitos em transporte público (ônibus, metrô e trem) e calcula, para cada trajeto, o CO2 economizado, os pontos ganhos e o cashback correspondente. O objetivo é incentivar o uso de transporte sustentável, oferecendo ao front-end um back-end completo para cadastrar, consultar, alterar e excluir esses cálculos.

O escopo desta entrega cobre o back-end: uma API desenvolvida em **Java 21 com Spring Boot**, persistência em **banco Oracle** via JDBC (padroes DAO e Connection Factory), regras de negócio e validações em uma camada BO, endpoints REST com status HTTP adequados, tratamento global de exceções, liberação de CORS para o front-end e publicação em nuvem.

2. Descrição da solução e principais funcionalidades

A solução segue uma arquitetura em camadas. O front-end envia apenas o nome do usuário, o tipo de transporte e a distância percorrida. A API valida os dados, calcula os valores e persiste o resultado, de modo que o cliente não consegue enviar CO2, pontos ou cashback falsos.

Funcionalidades implementadas

• **CRUD completo** de cálculos: criar, listar, buscar por id, alterar e excluir.

• **Cálculo automático** de CO2 economizado, pontos e cashback a partir do tipo de transporte e da distância.

• **Validações de negócio** (nome obrigatório, transporte valido, distância positiva e limitada). • **Normalização do transporte**: "Ônibus", "onibus" e "ONIBUS" são tratados como o mesmo valor. • **Tratamento de exceções** padronizado: toda falha retorna JSON com status, mensagem e horário. • **CorsFilter** para permitir que o front-end acesse a API de outra origem.

• **Deploy em nuvem** (Render) com credenciais do banco em variáveis de ambiente.

Regras de negócio

Regra	Valor
Taxa de CO2 por km - Ônibus	0,09 kg/km
Taxa de CO2 por km - Metrô	0,05 kg/km
Taxa de CO2 por km - Trem	0,06 kg/km
Pontos	3 pontos por km (parte inteira)
Cashback	R$ 0,01 por ponto
Distância	maior que 0 e no máximo 1000 km por trajeto
Nome do usuário	obrigatório, até 100 caracteres e não pode ser apenas numérico
Tipo de transporte	apenas Ônibus, Metrô ou Trem (com ou sem acento)

Exemplo: um trajeto de 10 km de ônibus gera 10 x 0,09 = 0,9 kg de CO2 economizado, 30 pontos e R$ 0,30 de cashback.

3. Tabela de endpoints (API RESTful)

Link da API publicada: `https://calculadoraco2.onrender.com/calculos`

Verbo	URI	Descrição	Status esperados
GET	/calculos	Lista todos os cálculos.	200 OK 500 erro de banco
GET	/calculos/{id}	Busca um cálculo pelo id.	200 OK 404 não encontrado 500 erro de banco
POST	/calculos	Cria um cálculo. Corpo: nome, tipoTransporte e distanciaKm. Retorna o objeto com o id gerado e o cabeçalho Location.	201 Created 400 dados inválidos 500 erro de banco
PUT	/calculos/{id}	Altera um cálculo existente e recalcula CO2, pontos e cashback.	200 OK 400 dados inválidos 404 não encontrado 500 erro de banco
DELETE	/calculos/{id}	Exclui um cálculo.	204 No Content 404 não encontrado 500 erro de banco

Também retornam **400** JSONs mal formados ou com tipos errados (por exemplo, texto no campo distanciaKm) e **404** qualquer URI inexistente.

Corpo de requisição (POST e PUT)

```
{
 "nome": "usuario",
 "tipoTransporte": "Onibus",
 "distanciaKm": 10
}
```

Resposta de sucesso (201 Created)

```
{
 "id": 1,
 "nome": "usuario",
 "tipoTransporte": "ONIBUS",
 "distanciaKm": 10.0,
 "co2Economizado": 0.9,
 "pontos": 30,
 "cashback": 0.3
}
```

Resposta de erro (padrao para 400, 404 e 500)

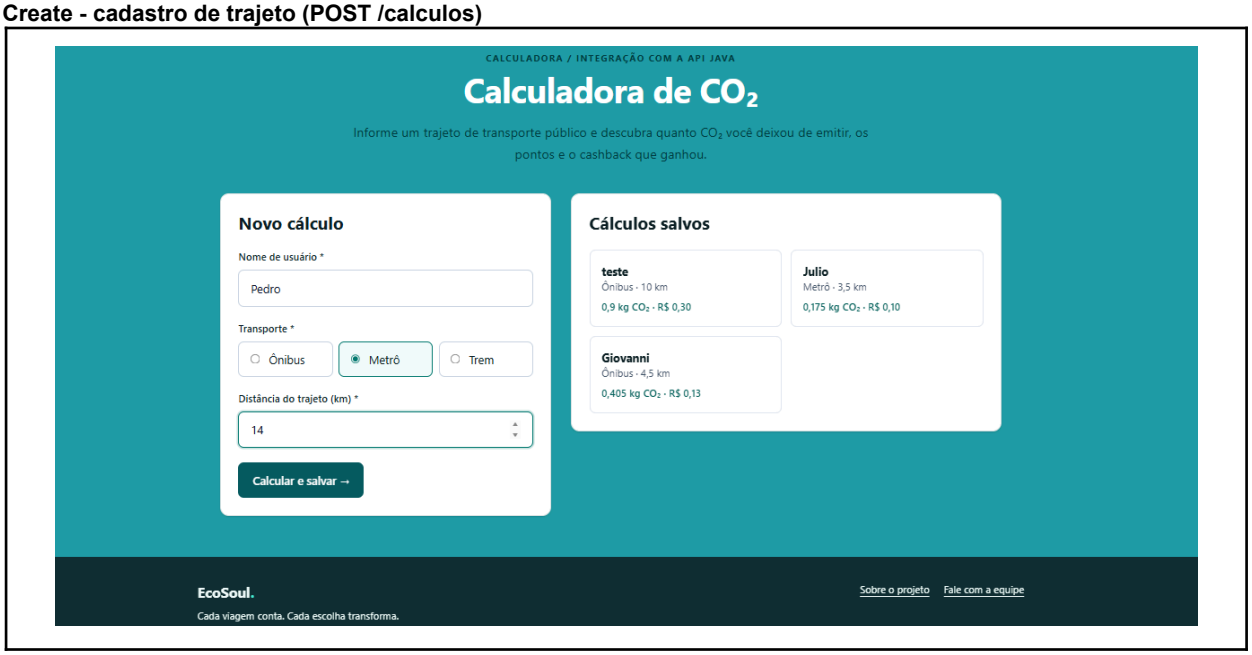
```
{
 "status": 400,
 "erro": "Bad Request",
 "mensagem": "A distância deve ser maior que zero.",
 "timestamp": "2026-10-29T15:41:10.82"
}
```

Em caso de erro de banco (500), o cliente recebe uma mensagem genérica e o detalhe técnico fica apenas no log do servidor, para não expor informações internas.

4. Protótipo - prints das telas implementadas

Capturas de tela reais do sistema com explicações resumidas das operações do CRUD.

Create - cadastro de trajeto (POST /calculos)



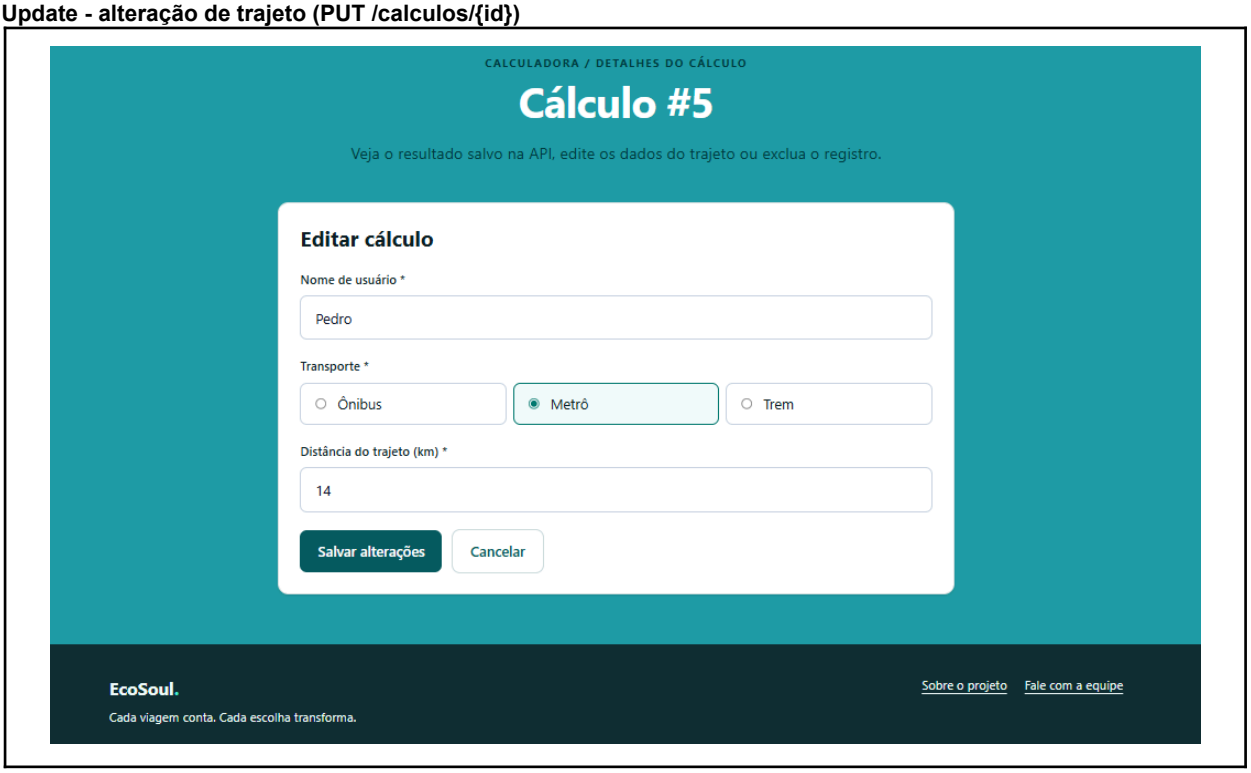
Explicação: O usuário informa nome, transporte e distância. A API valida os dados, calcula CO2, pontos e cashback, grava no Oracle e responde 201 Created com o id gerado.

Read - listagem e busca (GET /calculos e GET /calculos/{id})



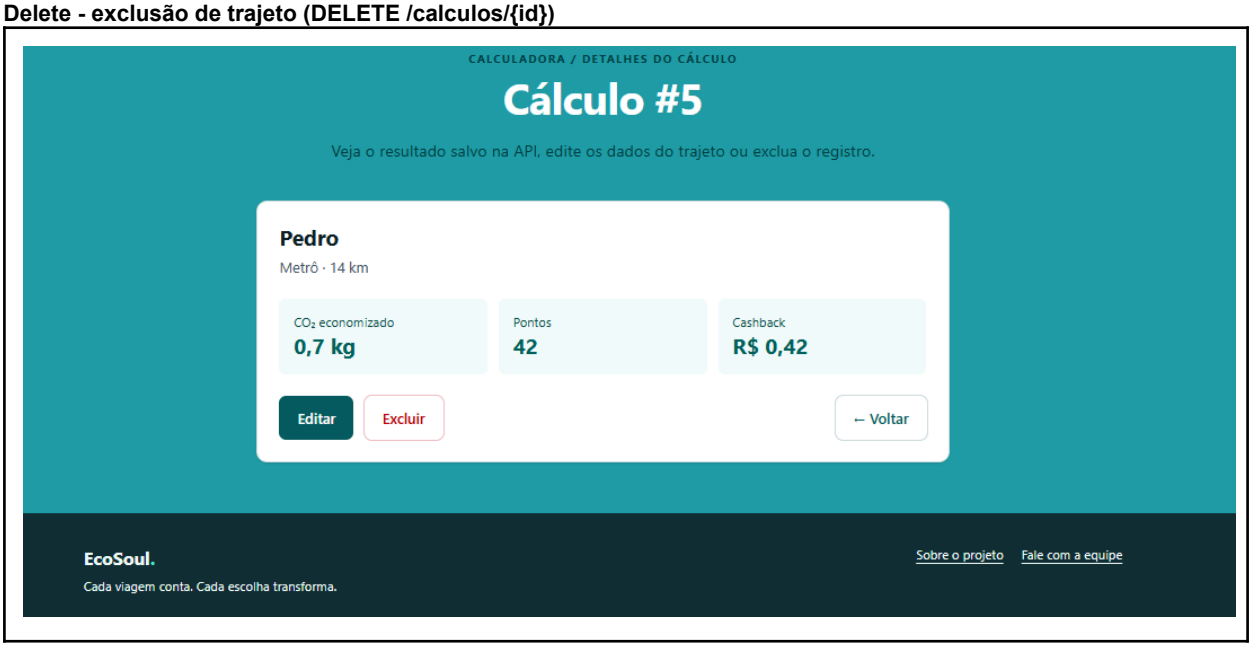
Explicação: Exibe os trajetos gravados com CO2 economizado, pontos e cashback. A busca por id retorna um único trajeto ou 404 quando ele não existe.

Update - alteração de trajeto (PUT /calculos/{id})



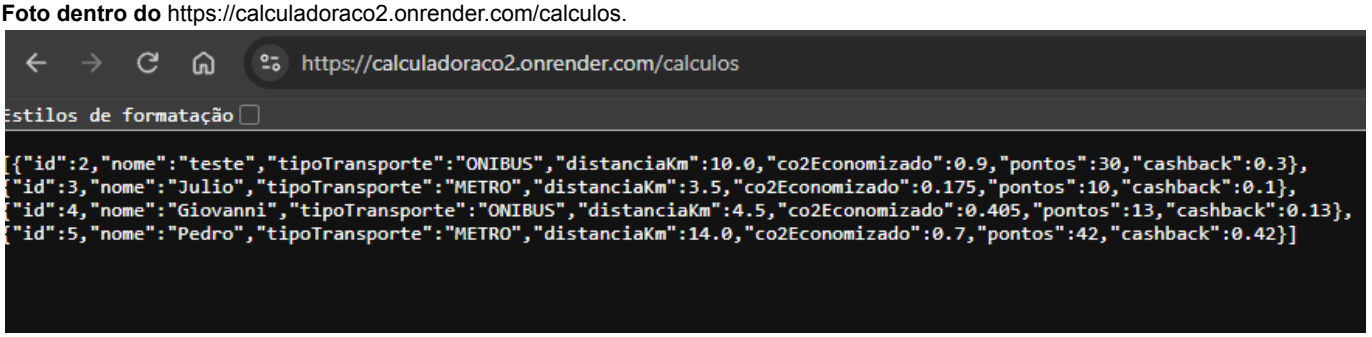
Explicação: Altera nome, transporte ou distância de um trajeto existente e recalcula CO2, pontos e cashback. Dados inválidos retornam 400 e id inexistente retorna 404.

Delete - exclusão de trajeto (DELETE /calculos/{id})



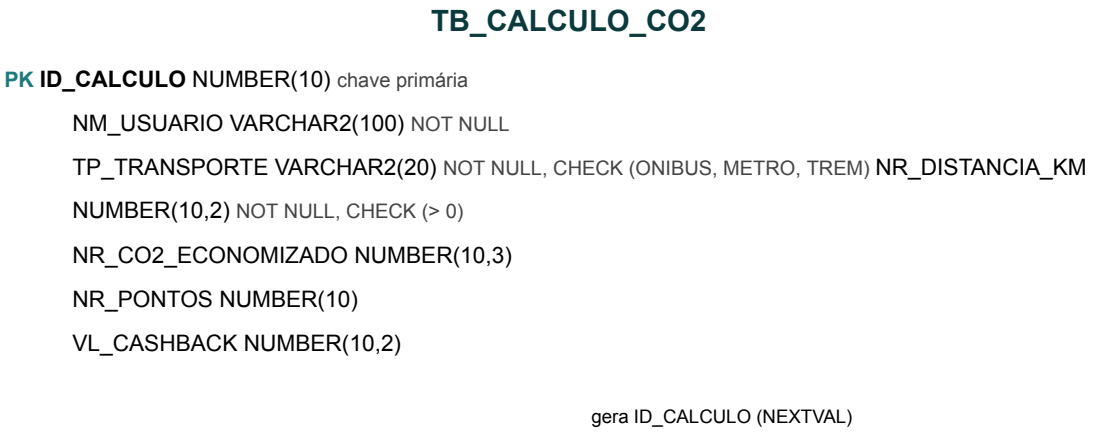
Explicação: Remove o trajeto do banco e responde 204 No Content. A listagem seguinte confirma que o registro não aparece mais.

Foto dentro do https://calculadoraco2.onrender.com/calculos.



5. Modelo Entidade-Relacionamento (MER)

O banco possui uma única entidade, **TB_CALCULO_CO2**, que armazena cada cálculo realizado. O identificador é gerado pela sequence **SEQ_CALCULO_CO2**. O tipo de transporte e controlado por uma constraint CHECK, portanto não há relacionamento com outras tabelas nesta versão.



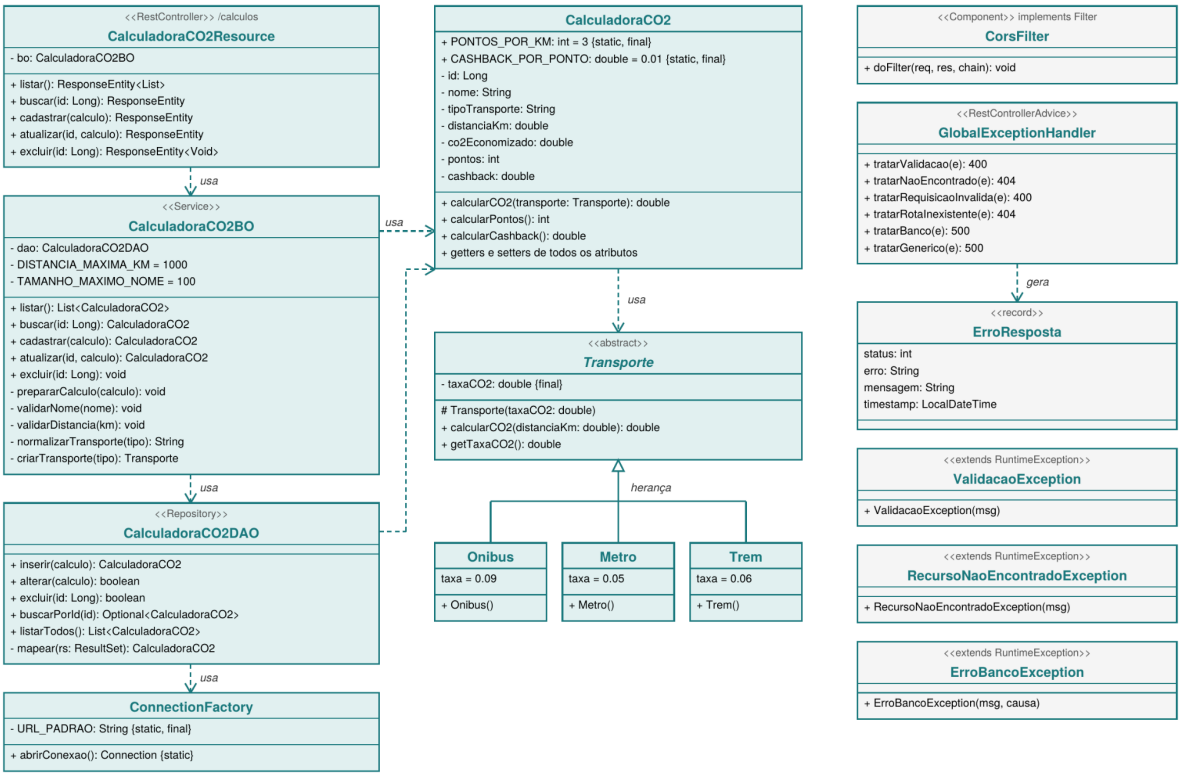
Coluna	Tipo	Restrições	Atributo Java
ID_CALCULO	NUMBER(10)	Chave primária (sequence)	id
NM_USUARIO	VARCHAR2(100)	NOT NULL	nome
TP_TRANSPORTE	VARCHAR2(20)	NOT NULL; CK_TP_TRANSPORTE	tipoTransporte
NR_DISTANCIA_KM	NUMBER(10,2)	NOT NULL; CK_DISTANCIA (> 0)	distanciaKm
NR_CO2_ECONOMIZADO	NUMBER(10,3)	calculado pelo BO	co2Economizado
NR_PONTOS	NUMBER(10)	calculado pelo BO	pontos
VL_CASHBACK	NUMBER(10,2)	calculado pelo BO	cashback

Script de criação

```
CREATE SEQUENCE SEQ_CALCULO_CO2 START WITH 1 INCREMENT BY 1;

CREATE TABLE TB_CALCULO_CO2 (
 ID_CALCULO NUMBER(10) PRIMARY KEY,
 NM_USUARIO VARCHAR2(100) NOT NULL,
 TP_TRANSPORTE VARCHAR2(20) NOT NULL,
 NR_DISTANCIA_KM NUMBER(10,2) NOT NULL,
 NR_CO2_ECONOMIZADO NUMBER(10,3),
 NR_PONTOS NUMBER(10),
 VL_CASHBACK NUMBER(10,2),
 CONSTRAINT CK_TP_TRANSPORTE CHECK (TP_TRANSPORTE IN ('ONIBUS','METRO','TREM')),
CONSTRAINT CK_DISTANCIA CHECK (NR_DISTANCIA_KM > 0)
);
```

6. Diagrama de classes atualizado



Relações entre as classes

Relação	Descrição
Resource -> BO -> DAO -> ConnectionFactory	Associação por injeção de dependência (construtor). Cada camada conhece apenas a seguinte.
BO e DAO usam CalculadoraCO2	Objeto de domínio que trafega entre as camadas.
CalculadoraCO2 usa Transporte	Dependência: o BO escolhe Onibus, Metro ou Trem (herança de Transporte) para calcular o CO2.
BO lança ValidacaoException e RecursoNaoEncontradoException	Quando os dados são inválidos (400) ou o id não existe (404).
DAO lança ErroBancoException	Quando ocorre falha de SQL ou de conexão (500).
GlobalExceptionHandler	Converte as exceções em ErroResposta (JSON) com o status HTTP correto.
CorsFilter	Filtro que libera o acesso do front-end e responde às requisições OPTIONS.

7. Procedimentos para rodar a aplicação

Ferramentas e versões

Item	Versão / observacao
IDE	IntelliJ IDEA (a equipe utilizou a versão 2025.3)
Java JDK	Java 21
Maven	Maven Wrapper incluido no projeto (mvnw); não precisa instalar o Maven
Framework	Spring Boot 3.5.6 (Spring Web)
Banco de dados	Oracle (servidor da FIAP) com driver ojdbc11
Controle de versão	Git e GitHub
Deploy	Render, com Docker (Dockerfile na raiz do projeto)
Testes de API	PowerShell (Invoke-RestMethod), Postman ou navegador

Como importar o projeto

1. Clone o repositório: `git clone https://github.com/EuThiaguera/CalculadoraCO2.git`
2. No IntelliJ, va em **File > Open** e selecione a pasta do projeto (a que contem o `pom.xml`). 3. Aguarde o Maven baixar as dependencias e confirme em **File > Project Structure** que o SDK e o Java 21.

Passo a passo da execução

1. No Oracle, execute o script da secao 5 para criar a sequence e a tabela.
2. Defina as variáveis de ambiente `DB_USER` e `DB_PASSWORD` (usuário e senha do Oracle). No IntelliJ: **Run > Edit Configurations**, selecione CalculadoraCo2Application e preencha **Environment variables** (uma por linha, ou separadas por ponto e vírgula).
3. Opcional: defina `DB_URL` para usar outra URL JDBC; por padrão a aplicação usa o Oracle da FIAP.
4. Execute a classe `CalculadoraCo2Application` pelo botão Run. A API sobe na porta 8080 (ou na porta definida pela variavel PORT).
5. Teste abrindo `http://localhost:8080/calculos`. O retorno `[]` indica que a API conectou no banco e a tabela está vazia.

Execução pelo terminal (alternativa)

```
$env:DB_USER = "seu_usuario"
$env:DB_PASSWORD = "sua_senha"
.\mvnw.cmd spring-boot:run
```

Deploy em nuvem

O projeto contem um `Dockerfile` com build em duas etapas (Maven e JRE 21). No Render, crie um Web Service a partir do repositório GitHub com o runtime Docker e cadastre `DB_USER` e `DB_PASSWORD` em Environment Variables. A aplicação publicada está em `https://calculadoraco2.onrender.com/calculos`. No plano gratuito, a primeira requisição após um período sem uso pode levar cerca de um minuto.